

EXPERIMENT 2

Introduction to C Programming – Area of a circle

- **Name:** Arushi Gusain
- **Date:**18/08/2026
- **Subject:** C programming

Aim

To write a C program to calculate the area of a circle, understand the stages of compilation using the -save-temps option, verify the generated intermediate files (.i, .s, .o), and execute the program in Linux.

Software Required

- Kali Linux
- GCC Compiler
- VI Editor
- Terminal

1. C Program to Find the Area of a Circle

File Name: area.c

```
#include<stdio.h>
```

```
#define PI 3.14
```

```
int main()
```

```
{
```

```
    float radius, area;
```

```
    printf("Enter the radius: ");
```

```
    scanf("%f", &radius);
```

```
    area = PI * radius * radius;
```

```
    printf("Area = %.2f\n", area);
```

```
    return 0;
}
```

Program Snippet:

1. C Program (area.c)

A screenshot of a text editor window titled 'kali@kali: ~/Desktop'. The editor shows a C program for calculating the area of a circle. The code is as follows:

```
1 #include<stdio.h>
2 #define PI 3.14
3
4 int main()
5 {
6     float radius, area;
7
8     printf("Enter the radius: ");
9     scanf("%f", &radius);
10
11     area = PI * radius * radius;
12
13     printf("Area = %.2f\n", area);
14
15     return 0;
16 }
```

The editor has a menu bar with 'File', 'Actions', 'Edit', 'View', and 'Help'. At the bottom, it shows '-- INSERT --' and a status bar with '16,2' and 'All'.

2. Stages of Compilation Using -save-temps

Command

```
gcc -save-temps area.c
```

This command generates the following files:

- area.i
- area.s
- area.o
- a.out

(a) Preprocessing Stage

Generated File: area.i

Command:

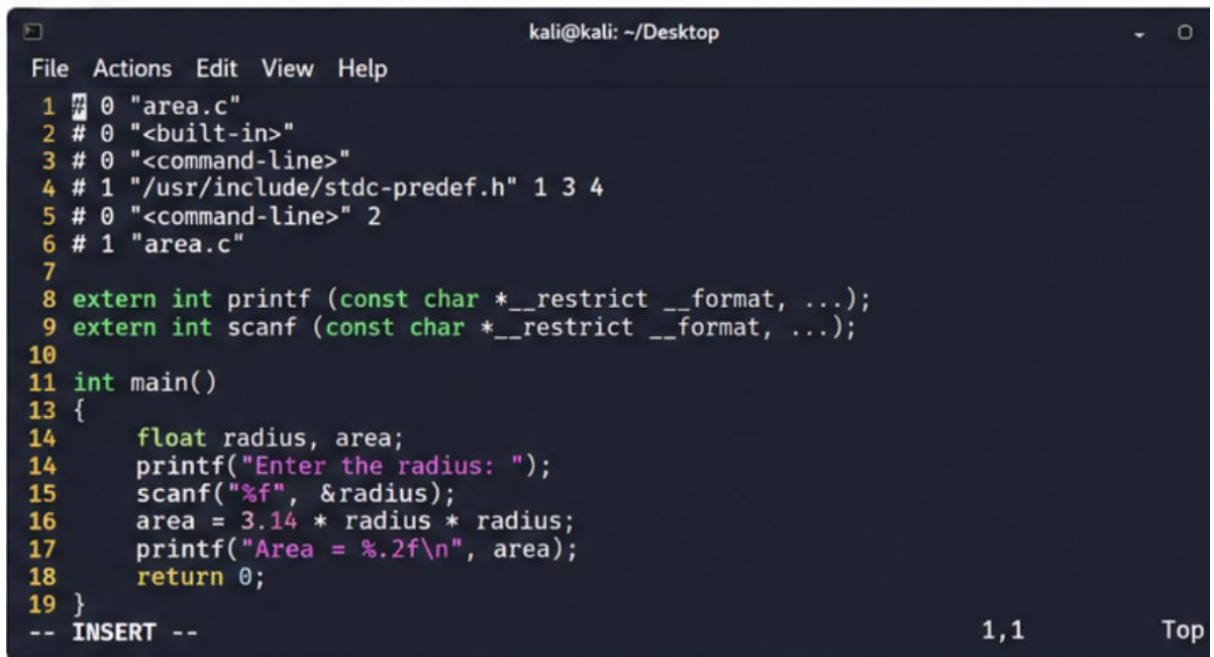
```
vi area.i
```

Explanation:

- Header files like stdio.h are expanded.
- Macro PI is replaced by 3.14.
- Comments are removed.
- The output is saved as area.i.

Snippet:

2. Preprocessed File (area.i)



```
File Actions Edit View Help
1 # 0 "area.c"
2 # 0 "<built-in>"
3 # 0 "<command-line>"
4 # 1 "/usr/include/stdc-predef.h" 1 3 4
5 # 0 "<command-line>" 2
6 # 1 "area.c"
7
8 extern int printf (const char *__restrict __format, ...);
9 extern int scanf (const char *__restrict __format, ...);
10
11 int main()
12 {
13     float radius, area;
14     printf("Enter the radius: ");
15     scanf("%f", &radius);
16     area = 3.14 * radius * radius;
17     printf("Area = %.2f\n", area);
18     return 0;
19 }
-- INSERT -- 1,1 Top
```

b) Compilation Stage

Generated File: area.s

Command:

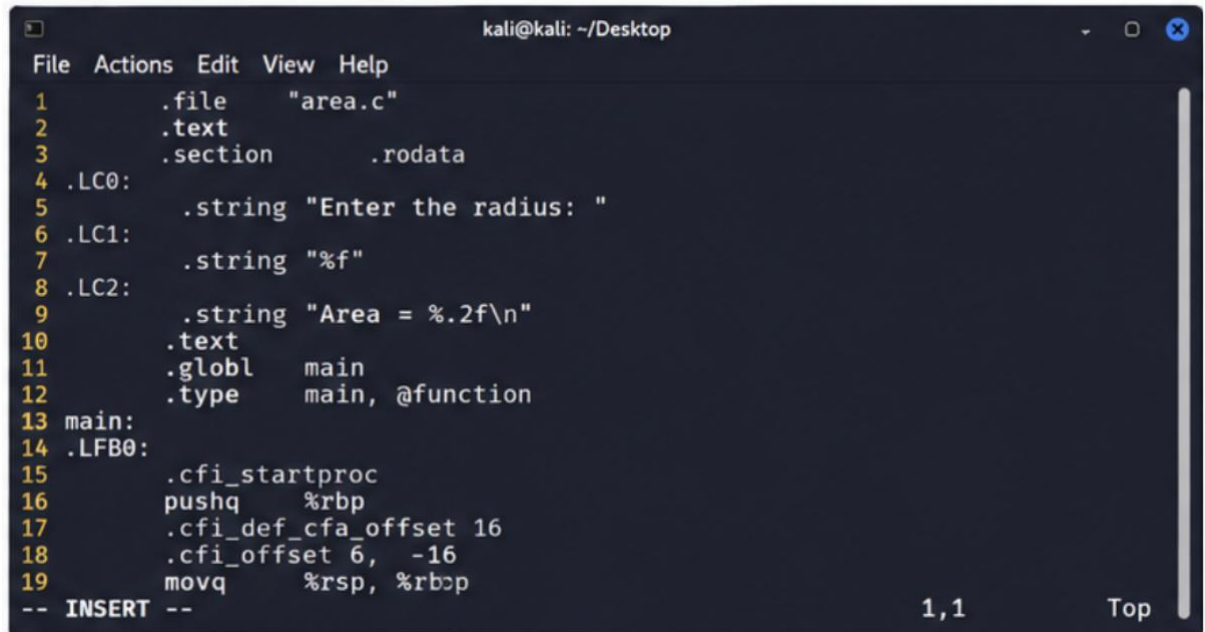
vi area.s

Explanation:

- The compiler converts the preprocessed C code into assembly language.
- The file contains CPU instructions in assembly format.

Snippet:

3. Assembly File (area.s)

A screenshot of a text editor window titled 'kali@kali: ~/Desktop'. The window displays assembly code for a file named 'area.s'. The code includes directives for file name, section, and string literals, followed by a function definition for 'main'. The code is as follows:

```
1 .file "area.c"
2 .text
3 .section .rodata
4 .LC0:
5 .string "Enter the radius: "
6 .LC1:
7 .string "%f"
8 .LC2:
9 .string "Area = %.2f\n"
10 .text
11 .globl main
12 .type main, @function
13 main:
14 .LFB0:
15 .cfi_startproc
16 pushq %rbp
17 .cfi_def_cfa_offset 16
18 .cfi_offset 6, -16
19 movq %rsp, %rbp
-- INSERT --
```

The editor interface includes a menu bar with 'File', 'Actions', 'Edit', 'View', and 'Help'. The status bar at the bottom right shows '1,1' and 'Top'.

(c) Assembly Stage

Generated File: area.o

Command:

vi area.o

Explanation:

- The assembler converts assembly code into object (machine) code.
- Since it contains binary data, the contents are mostly unreadable.

(d) Linking Stage

The linker combines the object file with required library files and creates the executable file.

Output:

a.out

3. Verifying Intermediate Files

View Preprocessed File

vi area.i

Observation:

- Human readable
- Contains expanded header files
- Macros are replaced

View Assembly File

vi area.s

Observation:

- Human readable
- Contains assembly language instructions

View Object File

vi area.o

Observation:

- Not human readable
- Displays binary/symbol characters

4. Program Execution

Compile the program:

gcc area.c

Run the executable:

`./a.out`

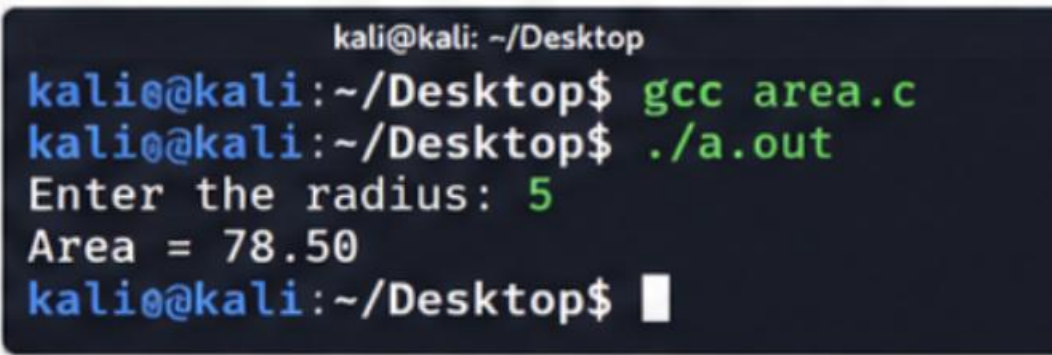
Sample Output

Enter the radius: 5

Area = 78.50

Execution Snippet:

5. Execution of Program (./a.out)

A terminal window screenshot with a dark background. The prompt is 'kali@kali: ~/Desktop'. The user enters 'gcc area.c' and presses enter. The prompt changes to 'kali@kali: ~/Desktop\$' and the user enters './a.out' and presses enter. The program prompts 'Enter the radius: ' and the user enters '5'. The program outputs 'Area = 78.50'. The prompt returns to 'kali@kali: ~/Desktop\$' with a cursor.

```
kali@kali: ~/Desktop
kali@kali:~/Desktop$ gcc area.c
kali@kali:~/Desktop$ ./a.out
Enter the radius: 5
Area = 78.50
kali@kali:~/Desktop$
```

Result

The C program to calculate the area of a circle was executed successfully. The stages of compilation were studied using the gcc -save-temps option. The generated files (area.i, area.s, area.o) were examined, and the executable program (a.out) produced the correct output.